

Yuanta Wealth — Supabase API Reference

This is the REST API for the Supabase PoC that mirrors Yuanta Wealth's HubDB tables. It's a **public, read-only, auto-generated REST API** (PostgREST, provided by Supabase) — every table listed below is queryable directly over HTTPS with no backend code needed. This doc is written so someone outside this project can pick it up and start pulling real data immediately.

Status: Proof-of-concept only. This is a sandboxed evaluation project, isolated from production HubSpot/HubDB. Data here is a snapshot, not live-synced.

Assets are fully self-hosted. Every `cover_image`, `photo`, `thumbnail`, and `pdf_url` field points at this project's own Supabase Storage bucket (`wealth-assets`), not HubSpot — the actual image/PDF files were migrated over, not just the row data. This PoC has zero runtime dependency on HubSpot.

1. Base URL & Auth

Base URL: `https://kqgdvpqygepvaifzrxki.supabase.co/rest/v1/`

Every request needs two headers — a publishable ("anon") key, safe to use in client-side code (browser JS, mobile apps). Row-Level Security is enabled on every table with a public **read-only** policy, so this key can `SELECT` but never `INSERT / UPDATE / DELETE` .

```
apikey: sb_publishable_6khmxt87r-YG1SxyF9d9XA_G0NNDTbp
Authorization: Bearer sb_publishable_6khmxt87r-YG1SxyF9d9XA_G0NNDTbp
```

curl example:

```
curl "https://kqgdvpqygepvaifzrxki.supabase.co/rest/v1/weekly_hot_issue?
select=main_title,page_date&limit=3" \
-H "apikey: sb_publishable_6khmxt87r-YG1SxyF9d9XA_G0NNDTbp" \
-H "Authorization: Bearer sb_publishable_6khmxt87r-YG1SxyF9d9XA_G0NNDTbp"
```

JavaScript (browser) example:

```
fetch('https://kqgdvpqygepvaifzrxki.supabase.co/rest/v1/weekly_hot_issue?
select=main_title,page_date&limit=3', {
  headers: {
    apikey: 'sb_publishable_6khmxt87r-YG1SxyF9d9XA_G0NNDTbp',
    Authorization: 'Bearer sb_publishable_6khmxt87r-YG1SxyF9d9XA_G0NNDTbp',
  },
}).then(r => r.json()).then(console.log);
```

2. Query syntax cheatsheet (PostgREST)

Every table endpoint (`/rest/v1/<table>`) accepts these query-string operators. Combine them freely with `&` .

Need	Syntax	Example
Pick columns	<code>select=col1,col2</code>	<code>?select=main_title,page_date</code>
Pick all columns	<code>select=*</code> (or omit <code>select</code>)	<code>?select=*</code>
Filter: equals	<code>col=eq.value</code>	<code>?week_slug=eq.2026-07-01</code>
Filter: not equal	<code>col=neq.value</code>	<code>?path=neq.2026-07-01-equity</code>
Filter: greater/less than	<code>col=gt.value</code> / <code>col=lt.value</code>	<code>?page_date=gt.2026-01-01</code>
Filter: in a list	<code>col=in.(a,b,c)</code>	<code>?model=in.(defender,growth)</code>
Sort	<code>order=col.asc</code> / <code>order=col.desc</code>	<code>?order=page_date.desc</code>
Limit rows	<code>limit=N</code>	<code>?limit=10</code>
Skip rows (pagination)	<code>offset=N</code>	<code>?limit=10&offset=10</code>
Combine filters (AND)	just chain with <code>&</code>	<code>?week_slug=eq.2026-07-01&limit=5</code>

Dates (`date` columns) are plain ISO strings (`"2026-07-01"`), not epoch milliseconds — this is a difference from raw HubDB row JSON, where dates are epoch-ms.

Full PostgREST reference: https://docs.postgrest.org/en/stable/references/api/tables_views.html

3. Row-identity conventions

Two different content shapes exist across these tables — this determines how you look up "the one row for this page":

- **Single-per-period tables** (e.g. `weekly_market_calendar` , `monthly_market_outlook`): one row per `week_slug` . Look up with `?week_slug=eq.<date>` .
- **Split tables** (e.g. `weekly_hot_issue` , `monthly_hot_issue` , `monthly_asset_class_outlook`): **multiple** topic rows can share one `week_slug` (one publish cycle covers several articles). Each row's unique identity is its `path` column (mirrors HubDB's `hs_path`) — look up a specific article with `?path=eq.<slug>` , or list every topic in a period with `?week_slug=eq.<date>` .

`week_slug` values are the **first day of the period**: `YYYY-MM-DD` for a week's Monday, or `YYYY-MM-01` for a month.

4. Tables

Weekly content

`weekly_hot_issue` — *split table*

Weekly "what's happening in markets" articles. Multiple topics per week.

Column	Type	Notes
<code>path</code>	text	unique row id, e.g. <code>2026-06-15-fed-rate-decision</code>
<code>week_slug</code>	text	e.g. <code>2026-06-15</code>
<code>main_title</code> , <code>page_subtitle</code> , <code>sub_title</code>	text	
<code>page_date</code>	date	
<code>description</code>	text	short excerpt (falls back to stripped <code>ai_summary</code> if empty)
<code>ai_summary</code>	text	HTML
<code>body</code>	text	HTML, full article
<code>cover_image</code>	jsonb	<code>{id, url, type}</code>

```
# All topics published in the week of 2026-06-15
curl ".../weekly_hot_issue?select=*&week_slug=eq.2026-06-15" -H "..."
# One specific article
curl ".../weekly_hot_issue?select=*&path=eq.2026-06-15-fed-rate-decision" -H "..."
```

weekly_asset_performance — one row per week

Weekly asset-class performance recap. `week_slug` , `main_title` , `page_subtitle` , `page_date` , `description` , `ai_summary` , `body` , `cover_image` (same shape as above).

weekly_buy_list — 3 rows per week (one per portfolio model)

Weekly recommended portfolio. `model` is `defender` / `flexible` / `growth` .

Column	Notes
<code>week_slug</code> , <code>model</code>	composite identity — one row per (week, model)
<code>main_title</code> , <code>page_subtitle</code> , <code>page_date</code> , <code>cio_content</code>	
<code>core_pct</code> , <code>satellite_pct</code>	numeric, portfolio split
<code>gfi_*</code> , <code>geq_*</code>	Core holdings: Global Fixed Income / Global Equity — <code>_weight</code> , <code>_lt_return</code> , <code>_target_12m</code> , <code>_eps_12m</code> , <code>_funds</code> (comma-separated TICKER:type list, type ∈ mf / dr / etf)
<code>sat_1_*</code> ... <code>sat_8_*</code>	up to 8 Satellite holdings, same suffix pattern as <code>gfi_*</code> / <code>geq_*</code> , plus <code>_sub</code> and <code>_name</code>
<code>perf_lt_return</code> , <code>perf_core_return</code> , <code>perf_sat_return</code> , <code>perf_risk_sd</code> , <code>perf_var_95</code>	headline performance/risk stats
<code>source_text</code>	footnote/disclaimer text

```
curl ".../weekly_buy_list?select=*&week_slug=eq.2026-06-15&model=eq.defender" -H "..."
```

weekly_market_calendar — one row per week

Upcoming economic calendar. Same base shape as `weekly_asset_performance` plus `week_start_date` / `week_end_date` .

weekly_pdf — one row per week

Just a pointer to the published PDF report.

Column	Notes
week_slug	
pdf_url	jsonb {id, url, type} — url is the direct PDF link

```
curl ".../weekly_pdf?select=week_slug, pdf_url&order=week_slug.desc&limit=1" -H "..."
```

Monthly content

Same shapes as their weekly counterparts, one publish cycle per calendar month (week_slug = 'YYYY-MM-01'):

Table	Shape	Notes
monthly_hot_issue	split (by path)	
monthly_asset_performance	one row/month	
monthly_market_outlook	one row/month	
monthly_market_calendar	one row/month	
monthly_asset_class_outlook	split (by path)	multiple asset-class deep-dives per month, e.g. 2026-07-01-equity , 2026-07-01-fixed-income , 2026-07-01-gold , 2026-07-01-global-fixed-income-portfolio
monthly_buy_list	3 rows/month (by model)	same column set as weekly_buy_list , but satellite holdings only go up to sat_8_* (not 12, despite older front-end code looping to 12 — columns 9–12 simply don't exist here)
monthly_pdf	one row/month	same shape as weekly_pdf

```
# All asset-class-outlook articles for July 2026
curl ".../monthly_asset_class_outlook?select=*&week_slug=eq.2026-07-01" -H "..."
```

Shared reference data

experts — analyst bios, used on every article page's "บทวิเคราะห์โดย" section

Column	Notes
name , title , credentials	
photo	jsonb {id, url, type}
order	numeric, sort ascending for display order

```
curl ".../experts?select=*&order=order.asc" -H "..."
```

fund_detail — lookup by ticker, used by the "recommended fund" drawer on buy-list pages

Column	Notes
ticker	e.g. ACWI , UGISFX-N — this is the lookup key , not id
fund_name , fund_type (mf / dr / etf)	
ai_summary	HTML, short AI blurb
content	HTML, fund highlights/detail

```
curl ".../fund_detail?select=*&ticker=eq.ACWI" -H "..."
```

events — Privileges & Events listing

slug , title , event_date , event_time , location , image (jsonb), body (HTML), category , is_published (boolean — filter with ?is_published=eq.true to match what the live site shows).

blog_general — generic promo/campaign blog posts

slug , main_campaign , campaign_value , title , thumbnail (jsonb), content (HTML), conditions_title / conditions (+ _1 / _2 variants for multiple T&C blocks), cta_label , cta_url .

Internal / not for production consumption

- preview_hot_issue , preview_asset_class_outlook — pre-publish staging duplicates used by the internal editorial workflow before content goes live. Same shape as their real counterparts. **Skip these**

unless you're specifically building an internal preview tool — they were intentionally excluded from the public-facing PoC page clones.

5. Common patterns

"Give me the latest published row" (used throughout the PoC front-end as the no-URL-param default):

```
curl ".../weekly_buy_list?select=week_slug&order=week_slug.desc&limit=1" -H "..."
```

"Give me everything published in one week/month, across categories" (related-articles pattern — call each table once, `week_slug=eq.<slug>`, and merge client-side):

```
curl ".../weekly_hot_issue?select=*&week_slug=eq.2026-06-15" -H "..."  
curl ".../weekly_asset_performance?select=*&week_slug=eq.2026-06-15&limit=1" -H "..."  
curl ".../weekly_buy_list?select=*&week_slug=eq.2026-06-15&limit=1" -H "..."  
curl ".../weekly_market_calendar?select=*&week_slug=eq.2026-06-15&limit=1" -H "..."
```

Rendering `ai_summary` / `body` / `content` fields: these are raw HTML strings (rich text from the original CMS) — render with `innerHTML`, don't escape them as plain text. If you need a plain-text excerpt, strip tags client-side (there's no separate plain-text column except `description`, which is sometimes empty).

6. Reference implementation

Every query pattern above is exercised in a real working page in this repo — see `sbFetch()` (the shared fetch helper) and its call sites in any `single-*.html` file, e.g. `single-hot-issue.html` (split-table lookup), `single-buy-list.html` (multi-model + fund-drawer lookup), or `single-monthly-buy-list.html` (cross-category related-articles fetch).